COL7880: Assignment 1 Report

January 28, 2026

1 Introduction

This report analyzes the parallelization strategies and performance results for a high-frequency stock trading engine simulation. The objective was to optimize three core functions—`updateDisplay`, `printOrderStats`, and `totalAmountTraded`—using OpenMP to achieve maximum speedup on a multi-core architecture.

The experiments were conducted on datasets ranging from 10^4 to 2×10^8 orders, testing scalability across 4, 8, 12, and 16 threads.

2 Performance Analysis

2.1 1. updateDisplay Performance

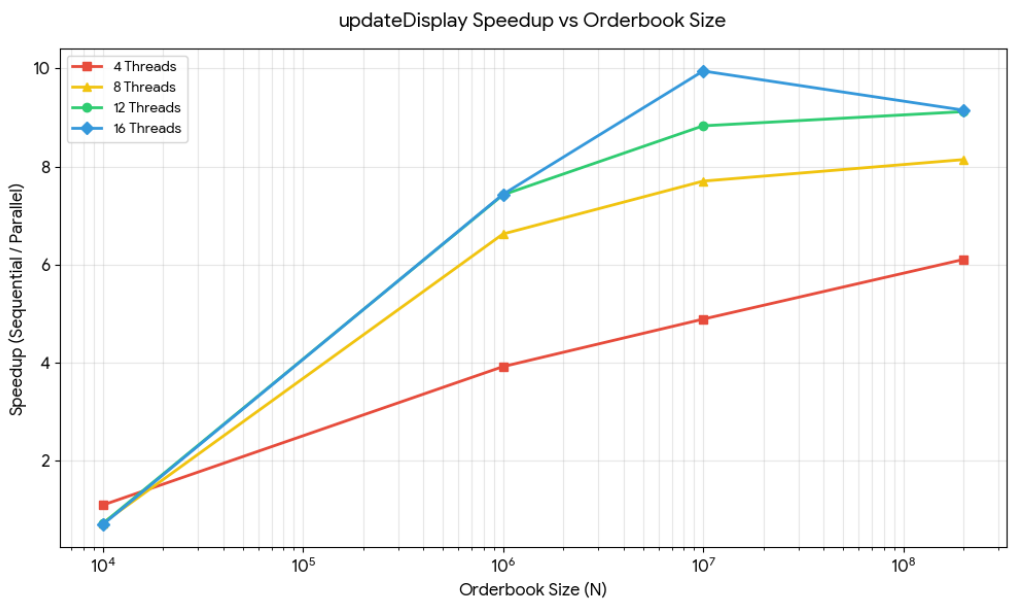

Figure 1: Speedup of updateDisplay vs. Input Size

Observation: The function achieves a stable speedup of approximately **9x** with 12-16 threads.

- **Scaling:** Performance scales well up to 12 threads but shows diminishing returns at 16 threads.
- **Bottleneck:** The plateau is attributed to the memory bandwidth required to copy the orderbook for each snapshot and the sequential nature of the underlying disk I/O subsystem.

2.2 2. totalAmountTraded Performance

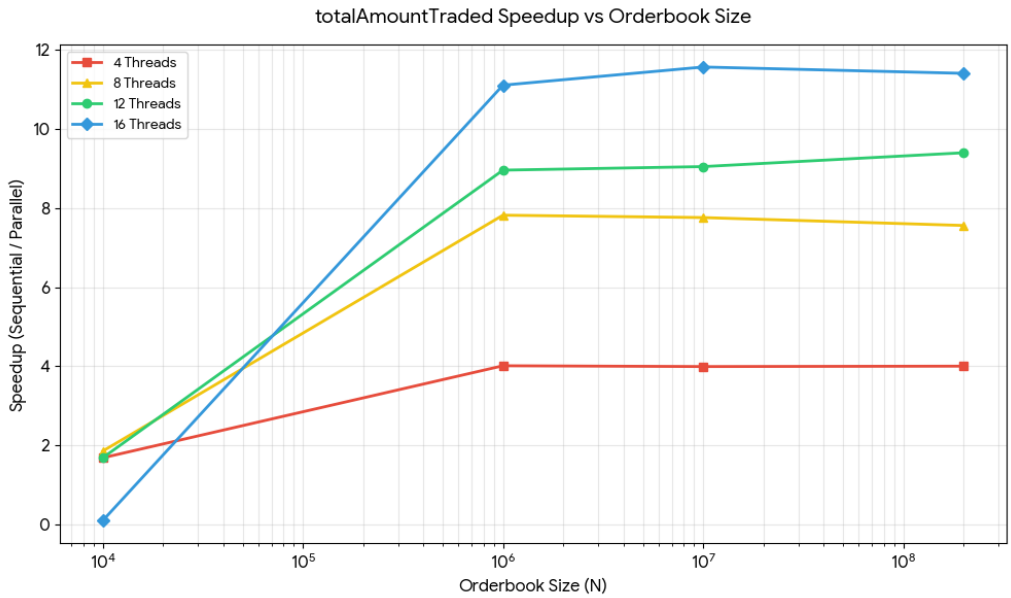

Figure 2: Speedup of totalAmountTraded vs. Input Size

Observation: This function demonstrates the most consistent scalability, reaching a peak speedup of **11.5x** with 16 threads.

- **Small Inputs:** At $N = 10^4$, speedup crashes (0.09x) because the overhead of spawning threads outweighs the trivial computation time.
- **Large Inputs:** For $N > 10^6$, the function is purely CPU-bound and scales linearly, confirming the efficiency of the OpenMP reduction.

2.3 3. printOrderStats Performance

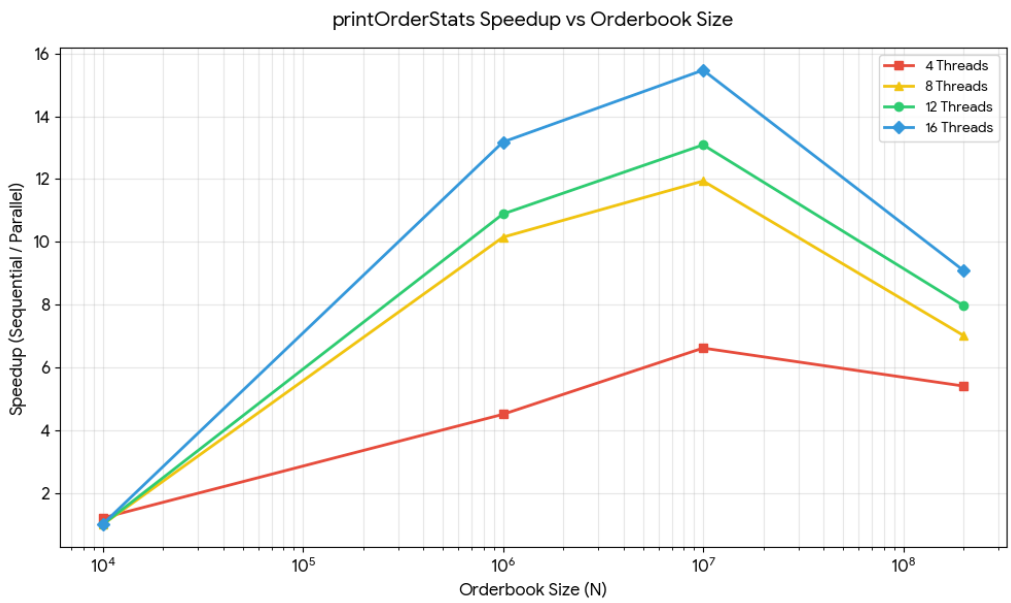

Figure 3: Speedup of printOrderStats vs. Input Size

Observation: The performance varies significantly based on the "Number of Stocks" parameter.

- **Peak Performance:** At $N = 10^7$ (10k stocks), the function achieves its maximum speedup of **15.47x** on 16 threads, showing near-perfect scaling.
- **The "Dip":** At $N = 2 \times 10^8$, the speedup drops to **9x**. This particular test case involved **100,000 stocks** (up from 10k). The increased number of unique keys inflated the cost of the "Merge Phase" (combining thread-local maps), which is the sequential bottleneck of this strategy.

Acknowledgments

- LLM used for debugging syntax issues, understanding sorting function, C++/OpenMP constraints, and plotting/collection help.
- Nitish (2025MCS2100), Monith (2025MCS2117)